

ASSIGNMENT – 2a

Multi-Client Banking Message Validation Using Python Socket Programming

AIM

To simulate a national bank's centralized transaction validation system where multiple ATM kiosks and branch terminals communicate with a validation server simultaneously. The server validates banking transaction requests (Account Verification, Balance Enquiry, Fund Transfer Validation, Transaction Confirmation) sent by the clients and echoes each request back as an acknowledgment.

Two client-server applications are developed in Python: a TCP-based version (1a) where the server accepts multiple clients concurrently using threads, and a UDP-based version (1b) where clients send transaction requests as datagrams. The TCP and UDP implementations are then compared (1c) on connection establishment, reliability, data ordering, and communication overhead. Communication continues until a client enters "exit", after which it disconnects without affecting the other connected clients.

1a. TCP – SERVER CODE

```
import socket
import threading

# socket module for network stuff, threading to handle many clients

def handle_client(conn, addr):
    print(f"Client Connected: {addr}")
    try:
        while True:
            data = conn.recv(1024).decode().strip()
            if not data:
                break

            print(f"[Client {addr[1]}] Received: {data}")

            if data.lower() == "exit":
                # client wants to leave, so we stop this thread
                break

            conn.sendall(data.encode())
            print(f"[Client {addr[1]}] Echo Sent: {data}")
    except ConnectionResetError:
        pass
    finally:
        print(f"Client {addr} disconnected.")
```

```
conn.close()

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
# allow fast restart of server on same port

server.bind(("127.0.0.1", 5000))
# listen only on this machine, port 5000

server.listen(5)
print("Banking Validation Server Started...")
print("Server listening on 127.0.0.1:5000")

while True:
    conn, addr = server.accept()
    # every client gets its own thread so they run at the same time
    t = threading.Thread(target=handle_client, args=(conn, addr), daemon=True)
    t.start()
```

1a. TCP – CLIENT CODE

```
import socket

# socket module to talk to the server

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(("127.0.0.1", 5000))

print("Connected to Banking Validation Server.\n")

while True:
    req = input("Enter Transaction Request: ")
    client.sendall(req.encode())

    if req.strip().lower() == "exit":
        # no reply expected, just leave
        print("Disconnected from Server.")
        break

    ack = client.recv(1024).decode()
    print("Server Acknowledgment:")
    print(ack)
```

```
client.close()
```

1a. TCP – SERVER OUTPUT

```
Banking Validation Server Started...
Server listening on 127.0.0.1:5000
Client Connected: ('127.0.0.1', 49254)
Client ('127.0.0.1', 49254) disconnected.
Client Connected: ('127.0.0.1', 49256)
Client Connected: ('127.0.0.1', 49257)
[Client 49256] Received: Balance Enquiry - ACC2005
[Client 49256] Echo Sent: Balance Enquiry - ACC2005
[Client 49257] Received: Account Verification - ACC1001
[Client 49257] Echo Sent: Account Verification - ACC1001
[Client 49256] Received: Mini Statement Request
[Client 49256] Echo Sent: Mini Statement Request
[Client 49257] Received: Transaction Confirmation - TXN45879
[Client 49257] Echo Sent: Transaction Confirmation - TXN45879
[Client 49256] Received: Fund Transfer Validation
[Client 49256] Echo Sent: Fund Transfer Validation
[Client 49257] Received: exit
Client ('127.0.0.1', 49257) disconnected.
[Client 49256] Received: exit
Client ('127.0.0.1', 49256) disconnected.
Client Connected: ('127.0.0.1', 49258)
[Client 49258] Received: exit
Client ('127.0.0.1', 49258) disconnected.
```

1a. TCP – CLIENT 1 OUTPUT

```
Connected to Banking Validation Server.

Enter Transaction Request: Account Verification - ACC1001
Server Acknowledgment:
Account Verification - ACC1001
Enter Transaction Request: Transaction Confirmation - TXN45879
Server Acknowledgment:
Transaction Confirmation - TXN45879
Enter Transaction Request: exit
Disconnected from Server.
```

1a. TCP – CLIENT 2 OUTPUT

```
Connected to Banking Validation Server.
```

```
Enter Transaction Request: Balance Enquiry - ACC2005
Server Acknowledgment:
Balance Enquiry - ACC2005
Enter Transaction Request: Mini Statement Request
Server Acknowledgment:
Mini Statement Request
Enter Transaction Request: Fund Transfer Validation
Server Acknowledgment:
Fund Transfer Validation
Enter Transaction Request: exit
Disconnected from Server.
```

1b. UDP – SERVER CODE

```
import socket

# UDP banking validation server, connectionless so no accept/threads needed

server = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
server.bind(("127.0.0.1", 5001))
print("UDP Banking Validation Server Started...")
print("Server listening on 127.0.0.1:5001")

while True:
    data, addr = server.recvfrom(1024)
    msg = data.decode().strip()
    print(f"[Client {addr[1]}] Received: {msg}")
    if msg.lower() == "exit":
        # udp has no connection to close, just ignore the exit request
        continue
    server.sendto(data, addr)
    print(f"[Client {addr[1]}] Echo Sent: {msg}")
```

1b. UDP – CLIENT CODE

```
import socket

# UDP client for banking transaction validation

client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

print("UDP Client ready (connectionless, just sending datagrams).\n")
```

```
while True:
    req = input("Enter Transaction Request: ")
    client.sendto(req.encode(), ("127.0.0.1", 5001))

    if req.strip().lower() == "exit":
        # nothing to close, udp has no connection
        print("Sent exit. No connection to terminate.")
        break

    ack, addr = client.recvfrom(1024)
    print("Server Acknowledgment:")
    print(ack.decode())

client.close()
```

1b. UDP – SERVER OUTPUT

```
UDP Banking Validation Server Started...
Server listening on 127.0.0.1:5001
[Client 58655] Received: Balance Enquiry - ACC2005
[Client 58655] Echo Sent: Balance Enquiry - ACC2005
[Client 58656] Received: Account Verification - ACC1001
[Client 58656] Echo Sent: Account Verification - ACC1001
[Client 58656] Received: Transaction Confirmation - TXN45879
[Client 58656] Echo Sent: Transaction Confirmation - TXN45879
[Client 58655] Received: Mini Statement Request
[Client 58655] Echo Sent: Mini Statement Request
[Client 58655] Received: Fund Transfer Validation
[Client 58655] Echo Sent: Fund Transfer Validation
[Client 58656] Received: exit
[Client 58655] Received: exit
[Client 58657] Received: exit
```

1b. UDP – CLIENT 1 OUTPUT

```
UDP Client ready (connectionless, just sending datagrams).

Enter Transaction Request: Account Verification - ACC1001
Server Acknowledgment:
Account Verification - ACC1001
Enter Transaction Request: Transaction Confirmation - TXN45879
Server Acknowledgment:
Transaction Confirmation - TXN45879
Enter Transaction Request: exit
Sent exit. No connection to terminate.
```

1b. UDP – CLIENT 2 OUTPUT

UDP Client ready (connectionless, just sending datagrams).

Enter Transaction Request: Balance Enquiry - ACC2005

Server Acknowledgment:

Balance Enquiry - ACC2005

Enter Transaction Request: Mini Statement Request

Server Acknowledgment:

Mini Statement Request

Enter Transaction Request: Fund Transfer Validation

Server Acknowledgment:

Fund Transfer Validation

Enter Transaction Request: exit

Sent exit. No connection to terminate.

1c. TCP vs UDP COMPARISON

Parameter	TCP	UDP
Connection Establishment	Connection-oriented: three-way handshake before data transfer and explicit connection close	Connectionless: datagrams are sent directly with no setup or teardown
Reliability	Reliable: guarantees delivery with acknowledgments and retransmission of lost packets	Unreliable: best-effort delivery, packets may be lost without detection
Data Ordering	Ordered: segments are sequenced and delivered in the same order they were sent	Unordered: datagrams may arrive out of order
Communication Overhead	Higher: larger headers, acknowledgments, connection state and flow control	Lower: small 8-byte header and no connection state to maintain

TASKS PERFORMED

Task	Details	Result
Create TCP server	Bind to 127.0.0.1:5000 and listen; spawn one thread per accepted client	Banking Validation Server Started; two clients connected at the same time
TCP echo validation	Server receives a transaction request, displays client IP/port, and echoes it back	Each client received its original request as acknowledgment
Handle client exit	Client sends 'exit' and disconnects; server keeps serving the other client	Client 1 left, Client 2 continued and completed its requests

Task	Details	Result
Create UDP server	Bind UDP socket to 127.0.0.1:5001; receive datagrams with <code>recvfrom</code> and echo with <code>sendto</code>	UDP Banking Validation Server Started; sender IP and port shown for every datagram
UDP echo validation	Two UDP clients send transaction requests as datagrams and receive echoes	Both clients received acknowledgments for every request
UDP exit behaviour	Client sends 'exit' and simply stops; no connection to terminate	Client printed 'No connection to terminate'; server kept running
Compare TCP and UDP	Tabulated both protocols on connection establishment, reliability, ordering, and overhead	TCP is reliable/ordered with higher overhead; UDP is faster and simpler but best-effort

LEARNING OUTCOMES

- I implemented a TCP server that serves multiple banking clients concurrently using Python threads.
- I understood that TCP requires `connect()`/`accept()` and a three-way handshake before data transfer.
- I implemented a UDP server and clients using `sendto()`/`recvfrom()` without any connection setup.
- I observed that the TCP server keeps serving other clients even after one client disconnects.
- I compared TCP and UDP on reliability, ordering, and overhead, and saw why banks use TCP for transaction validation.
- I learned that messages must be encoded to bytes before sending and decoded after receiving in both protocols.